

CONFIDENTIAL

Assessment Date
March 22,
2026

 VAPT REPORT

Vulnerability Assessment Report

Website Security Audit

Altoro Mutual (demo.testfire.net)

Prepared By

GANDONOU Koffi Patrick-Léon
Cyber Security Intern - Future Intern

Target

demo.testfire.net
65.61.137.117

Assessment Type

Passive VAPT
Read-Only



Section 01

Executive Summary

Q Assessment Overview

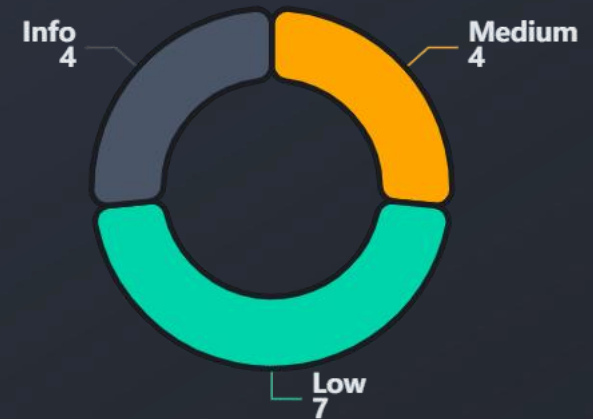
This report presents the findings of a **passive web application vulnerability assessment** conducted against Altoro Mutual (demo.testfire.net), a publicly authorized test banking platform maintained by HCL Technologies.

The assessment identified **15 alerts** across four risk tiers. The findings reveal a web application running on **highly outdated server infrastructure** and lacking critical browser-enforced security headers.

⚠ Key Message for Business

- ✗ This website is **operational but not secure** by modern standards.
- ✗ The issues identified are types that **attackers routinely exploit** against banking applications.
- ✓ Fixing them is **not expensive** — it primarily requires configuration changes and a server upgrade.

Risk Distribution



0

High Risk

4

Medium Risk

7

Low Risk

4

Informational



Section 02

Scope & Rules of Engagement

🎯 Assessment Parameters



Target Website

<https://demo.testfire.net>



IP Address

65.61.137.117



Authorization

Public test app — HCL Technologies



Assessment Type

Read-Only / Passive

🚫 Out of Scope

- ✗ Any active exploitation attempts
- ✗ Denial-of-service (DoS) tests
- ✗ Credential attacks or brute force
- ✗ Social engineering attacks
- ✗ Physical security testing

✅ In Scope Activities

- 1 **Reconnaissance & Information Gathering**
Open-source intelligence and surface mapping
- 2 **Port & Service Enumeration**
Nmap scanning for open ports and services
- 3 **Technology Fingerprinting**
Identifying frameworks, server versions, stack
- 4 **Automated Vulnerability Scanning**
Passive scanning with OWASP ZAP and Nikto
- 5 **Security Header Analysis**
Checking for missing or misconfigured headers

i Ethical Note

This assessment strictly adhered to a **read-only, non-destructive approach**. No data was accessed, modified, or exfiltrated during the assessment.



Section 03

Tools & Methodology

Tools Used



Nmap v7.95

Port scanning, OS detection, service enumeration



Nikto v2.5.0

Web server vulnerability scanner



WhatWeb

Technology fingerprinting & framework detection



OWASP ZAP v2.16.1

Automated passive web app vulnerability scanner



Kali Linux

Operating system for security assessments



Firefox Browser

Manual inspection and screenshot capture

Assessment Methodology

1

Reconnaissance

Initial open-source intelligence gathering to understand application surface area and confirm authorization.

2

Port & Service Enumeration (Nmap)

Identified open ports and running services including exact software versions.

3

Technology Fingerprinting (WhatWeb)

Confirmed Java/Apache Tomcat stack, session cookies, and HTTP header details.

4

Web Vulnerability Scan (Nikto)

Detected missing security headers and dangerous HTTP methods.

5

Application-Level Passive Scan (OWASP ZAP)

Passive automated analysis identifying 15 alert categories without exploitative requests.



Sample Command: `nmap -sV -sC -T4 -O demo.testfire.net`



Findings Summary

#	Vulnerability	Severity	Confidence	Found By
01	Absence of Anti-CSRF Tokens	Medium	High	OWASP ZAP
02	Content Security Policy (CSP) Header Not Set	Medium	High	OWASP ZAP
03	Missing Anti-Clickjacking Header (X-Frame-Options)	Medium	Medium	Nikto + ZAP
04	Dangerous HTTP Methods Enabled (PUT, DELETE)	Medium	Low	Nikto
05	Server Version Disclosed in HTTP Header	Low	High	Nmap + Nikto + ZAP
06	Outdated Server Component (Apache Tomcat Coyote 1.1)	Low	High	Nmap + WhatWeb
07	Strict-Transport-Security (HSTS) Header Not Set	Low	High	OWASP ZAP
08	X-Content-Type-Options Header Missing	Low	Medium	Nikto + ZAP
09	Cookie Without SameSite Attribute	Low	High	OWASP ZAP
10	Cross-Domain JavaScript Source File Inclusion	Low	Low	OWASP ZAP
11	Secure Pages Include Mixed Content	Low	Medium	OWASP ZAP
12	Information Disclosure — Suspicious Comments	Info	Medium	OWASP ZAP
13	Session Management Response Identified	Info	Medium	OWASP ZAP
14	Re-examine Cache-Control Directives	Info	Medium	OWASP ZAP
15	Modern Web Application Detection	Info	Medium	OWASP ZAP

0

High Risk

4

Medium Risk

7

Low Risk

4

Informational



Section 05

Critical Findings — Part 1

01

FINDING-01

Absence of Anti-CSRF Tokens

MEDIUM

What is the issue?

HTML forms (login, feedback, subscription) do not use Anti-CSRF (Cross-Site Request Forgery) tokens.

Business Impact

An attacker could embed a hidden form on a malicious website. When a logged-in customer visits, their browser silently submits a forged banking request (e.g., fund transfer) — **without the customer ever knowing**.

Remediation

Implement per-session, server-validated CSRF tokens on every form. Modern frameworks include this natively (e.g., Spring Security's `_csrf` token).

OWASP: A01:2021 – Broken Access Control URLs: /login.jsp, /feedback.jsp, /subscribe.jsp

02

FINDING-02

CSP Header Not Set

MEDIUM

What is the issue?

No Content-Security-Policy HTTP header is returned in any server response.

Business Impact

CSP is the primary browser-level defence against XSS attacks. Without it, if an attacker finds an XSS flaw, there is **no secondary barrier** to prevent malicious script execution that could steal session cookies and account data.

Remediation

Configure the web server to return: `Content-Security-Policy: default-src 'self'; script-src 'self'`. Start restrictive and gradually relax.

OWASP: A05:2021 – Security Misconfiguration Scope: All pages on demo.testfire.net



Section 05

Critical Findings — Part 2

03

FINDING-03

Missing Anti-Clickjacking Header

MEDIUM

What is the issue?

The X-Frame-Options HTTP header is absent from all server responses, confirmed by both Nikto and OWASP ZAP.

Business Impact

Attackers can embed this banking site inside an invisible one on a fake page. A user clicking "Play Video" may actually be clicking "Confirm Transfer" on the hidden banking site. This is called **Clickjacking** and is particularly dangerous for financial applications.

Remediation

Add X-Frame-Options: SAMEORIGIN or DENY header via Apache Tomcat config, or use CSP: frame-ancestors 'self'.

OWASP: A05:2021 – Security Misconfiguration Scope: All pages

04

FINDING-04

Dangerous HTTP Methods Enabled

MEDIUM

What is the issue?

The OPTIONS request returns allowed HTTP methods including **PUT and DELETE**, typically used to write or remove files.

Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS

Business Impact

If authorization is insufficient, an attacker could upload a malicious script ("web shell") via PUT, gaining **full remote code execution**, or delete critical files causing service outage.

Remediation

In Apache Tomcat config, restrict allowed HTTP methods to only GET and POST. Reject all others with a **405 Method Not Allowed** response.

Confidence: Low Found By: Nikto



Section 05

Critical Findings — Part 3

05

FINDING-05

Server Version Disclosed

LOW

What is the issue?

The exact web server software and version is openly broadcast in every HTTP response header.

Server: Apache-Coyote/1.1

Business Impact

Apache-Coyote/1.1 is a component from Apache Tomcat's **legacy architecture (15+ years old)**. Publishing this version allows any attacker to immediately search for known, public CVE exploits targeting this exact version — dramatically reducing attack staging time.

Remediation

Configure the server to suppress or genericize the Server header. Additionally, strongly prioritize an **urgent upgrade** to a modern, actively maintained Tomcat release.

Confidence: High

06

FINDING-06

HSTS Not Set

LOW

What is the issue?

The application does not return an HTTP Strict-Transport-Security (HSTS) header.

Business Impact

Even though the site supports HTTPS, a user typing the URL may first connect via HTTP before being redirected. An attacker on a shared network (coffee shop, airport Wi-Fi) can intercept this initial unencrypted request and strip SSL encryption for the session (**SSL strip attack**), allowing them to eavesdrop on account credentials.

Remediation

Configure the web server to return: Strict-Transport-Security: max-age=31536000; includeSubDomains

Confidence: High

07

FINDING-07

X-Content-Type-Options Missing

LOW

What is the issue?

The X-Content-Type-Options header is absent from server responses.

Business Impact

Without this header, browsers may "sniff" the MIME type of content and interpret files differently than declared. An attacker who can upload a file (e.g., an image containing script code) could trick a browser into executing it as JavaScript, enabling an **XSS-style attack vector**.

Remediation

Add X-Content-Type-Options: nosniff to all server HTTP responses.

Confidence: Medium



Section 05

Additional Findings (Low/Informational)

08 Cookie Without SameSite Attribute

Low

Session cookie JSESSIONID lacks SameSite=Strict, enabling CSRF via cross-origin requests. Add SameSite=Strict to Set-Cookie.

09 Cross-Domain JS Inclusion

Low

External JavaScript files loaded from third-party domains. If those domains are compromised, attacker JS runs on the banking site. Review and self-host critical scripts.

10 Secure Pages with Mixed Content

Low

HTTPS pages loading some resources over HTTP, weakening the security guarantee of HTTPS. Enforce HTTPS for all assets.

11 Suspicious Comments in Source Code

Info

HTML comments contain developer notes (e.g., keywords like TODO, internal paths). Strip all comments from production code.

12 Session Management Identified

Info

Session token (JSESSIONID) is visible and transmitted in cookies. Ensure token rotation on login/logout.

13 Cache-Control Issues

Info

Sensitive pages do not prevent caching, meaning browsers/proxies might cache banking page contents. Apply Cache-Control: no-store on authenticated pages.

14 Re-examine Cache-Control Directives

Info

Additional cache-control headers may need review to ensure sensitive data is not cached by intermediate proxies or browsers.

15 Modern Web Application Detection

Info

ZAP informational alert confirming the app uses modern frameworks. No action required — this is a positive finding indicating the application is built with contemporary technologies.



Note: These findings, while lower in severity, still contribute to the overall attack surface and should be addressed as part of a comprehensive security hardening program.

7
Low

4
Info



Technical Evidence — Scan Results

Nmap Port Scan Results

```
$ nmap -sV -sC -T4 -O demo.testfire.net
PORT STATE SERVICE VERSION
80/tcp open http Apache Tomcat/Coyote JSP engine 1.1
443/tcp open ssl/http Apache Tomcat/Coyote JSP engine 1.1
8080/tcp open http-proxy Open proxy behavior detected
```

- ✔ Port 80 (HTTP): Standard web traffic
- ✔ Port 443 (HTTPS): SSL/TLS encrypted traffic
- ⚠ Port 8080: Open proxy behavior detected — potential security risk

Technology Stack Identified

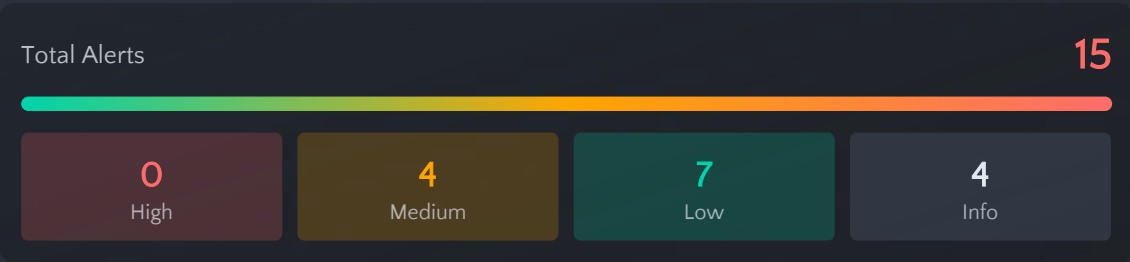
Web Server Apache Tomcat Coyote JSP 1.1	Platform Java/JSP Legacy Architecture
Session Cookie JSESSIONID No SameSite attr	Server Age 15+ Years Old Critical

Nikto Vulnerability Scan

```
$ nikto -h https://demo.testfire.net
+ Target IP: 65.61.137.117
+ Target Hostname: demo.testfire.net
+ + Allowed HTTP Methods: PUT, DELETE
+ + Missing X-Frame-Options Header
+ + Missing X-Content-Type-Options
+ + Server banner reveals version
+ + SSL v2/v3 supported (deprecated)
```

- ✗ Dangerous HTTP methods (PUT, DELETE) enabled
- ✗ Missing critical security headers (X-Frame-Options, X-Content-Type-Options)
- ⚠ Server version disclosure in banner
- ⚠ Deprecated SSL versions still supported

OWASP ZAP Passive Scan





Section 07

Conclusion & Security Rating



Overall Security Rating MODERATE RISK

The application is operational but running with significant security debt.

! Key Concerns

- 1 Absence of essential HTTP security headers exposes users to client-side attacks
- 2 Decade-old server component (Apache Tomcat Coyote 1.1) has known vulnerabilities
- 3 Platform exposed to well-documented, easily executed attacks against user base

📄 Assessment Summary

The Altoro Mutual application is **operational but is running with significant security debt**. The absence of essential HTTP security headers and the use of a decade-old server component expose the platform to well-documented, easily executed attacks — particularly against its user base.

15

Total Findings

4

Medium Risk

0

High Risk

💡 Positive Aspects

- ✓ No high-severity exploitable vulnerabilities detected within read-only scope
- ✓ Application uses HTTPS encryption for secure communications
- ✓ Modern web application frameworks detected (potential for security improvements)

Key Takeaway



While no critical vulnerabilities were found in the passive assessment, the **configuration weaknesses and outdated infrastructure** create a concerning security posture that should be addressed promptly.



Priority Remediation Plan

Priority	Action	Effort	Impact
P1	Upgrade Apache Tomcat to latest LTS versionCritical infrastructure update to address known CVEs	High	Critical
P1	Implement CSRF tokens on all formsAdd per-session, server-validated tokens to login, feedback, subscription forms	Medium	High
P2	Add Content-Security-Policy headerConfigure CSP to prevent XSS attacks: default-src 'self'; script-src 'self'	Low	High
P2	Add X-Frame-Options: SAMEORIGIN headerPrevent clickjacking attacks by restricting iframe embedding	Low	High
P3	Add Strict-Transport-Security headerEnable HSTS to prevent SSL strip attacks: max-age=31536000; includeSubDomains	Low	Medium
P3	Add X-Content-Type-Options: nosniff headerPrevent MIME type sniffing attacks	Low	Medium
P4	Disable PUT / DELETE HTTP methodsRestrict to GET and POST only; return 405 for others	Low	Medium
P4	Add SameSite=Strict to session cookiesEnhance CSRF protection for session management	Low	Low-Med

P1 (Critical)

2 Actions

P2 (High)

2 Actions

P3-P4 (Medium/Low)

4 Actions